DEPI — DIGITAL EGYPT PIONEERS INITIATIVE · DATA ENGINEERING TRACK

Web-to-Warehouse Product Intelligence Pipeline

Project Documentation — Parts 1 to 3 (Full consolidated deliverable)

Team Leader: Ahmed Abobakr (78102) — @Ahmedbakr78

GitHub: https://github.com/Ahmedbakr78/Web-to-Warehouse-Product-Intelligence-Pipeline

Version 1.10.0 · 2026-10-16 · 585 backend tests · 216 documented operations · 29 tables · 23 views · DQ score 98.26 on PostgreSQL and MySQL

Table of contents

Web-to-Warehouse Product Intelligence Pipeline 3
Team Leader 3
Project registration record 3
Document metadata 3
How this document was produced 4
1. Project Planning and Management 5
1.1 Project idea 5
1.2 Objectives (SMART) 5
1.3 Scope 6
1.4 Project plan (12 weeks) 6
1.5 Task assignment and responsibilities 8
1.6 Risk assessment and mitigation plan 8
1.7 KPIs (Key Performance Indicators) 10
2. Literature Review 11
2.1 Review method 11
2.2 Theme 1 — Dimensional modelling and the data warehouse 11
2.3 Theme 2 — Web crawling compliance, ethics and law 11
2.4 Theme 3 — Record linkage and entity resolution 12
2.5 Theme 4 — Data quality 12
2.6 Theme 5 — Data integration and orchestration 13
2.7 Comparison with similar existing solutions 13
2.8 Research gap and what this project adds 14
2.9 Documented sources (principal references) 14
3. Requirements Gathering 15
3.1 Elicitation techniques 15
3.2 Stakeholder analysis 15
3.3 User stories with acceptance criteria (selected) 17
3.4 Functional requirements (catalogue, representative and grouped) 19
3.5 Non-functional requirements 21
3.6 Business rules 22
3.7 Traceability (summary) 22
Appendix — How to verify every claim in this document 23
Appendix — All diagrams 24

Web-to-Warehouse Product Intelligence Pipeline

Project Documentation

Team Leader

Field	Value
Team Leader	Ahmed Abobakr
GitHub account	@Ahmedbakr78
Role	Team Lead, Backend and API platform, Documentation, 9 ETL stages owner
Effort weight	28% of the team effort (remaining 72% split across the five team members, section 1.5)
Owned components	FastAPI platform (23 routers, 216 documented operations), 9-stage ETL shared by CLI, REST API and Airflow, integration transport, catalogues as code, documentation set (32 documents), releases and quality gates
Team	Six members in total, one single repository for the whole team (section 1.5)

Project registration record

Item	Value
Project name (chosen)	Web-to-Warehouse Product Intelligence Pipeline
Project-specific GitHub repository (one link for the whole team)	https://github.com/Ahmedbakr78/Web-to-Warehouse-Product-Intelligence-Pipeline (repository visibility: Public)
Google Drive projects file registration	The project name and the GitHub link are to be entered by the team manually in the dedicated columns of the DEPI projects file on Google Drive; every artifact this document mentions is found under the single GitHub link above

Document metadata

Field	Value
DEPI track	Data Engineering — Digital Egypt Pioneers Initiative
GitHub repository	https://github.com/Ahmedbakr78/Web-to-Warehouse-Product-Intelligence-Pipeline (Public)
Delivered version	1.10.0
Documentation date	2026-10-16
Documentation source	This document consolidates and reflects the real, measured contents of the delivered repository (docs 00–31, app/, dags/, db/, tests/)

How this document was produced

This document was written after a full study of the delivered application, not from memory or estimation. Every figure below is taken from the project's measured statistics file (`docs/stats.json`), the repository modules, and the verification logs generated by `./run.sh`. The delivered system passed the full automated check on the date above:

```
./run.sh --verify-only
FULL APP TEST: PASS   (mode: REAL — live permitted sources, no fake data)
  backend lint (ruff)                    passed
  backend tests                          585 passed
  live-source smoke (3 permitted sources) passed, 45 live records
  real pipeline (extract-clean-dedupe-load-DQ-report) passed, status success
  frontend unit tests (vitest)           47 passed
  frontend production build              passed
  docker compose definition (6 services)  valid
```

System of record — `docs/stats.json` (v1.10.0):

Metric	Measured value
Catalogued features	550 (46 feature areas, served by GET /api/v1/meta/features)
REST route decorators / documented paths	216 decorators over 191 paths, across 23 routers
Physical tables / analytical SQL views	29 tables / 23 views (PostgreSQL and MySQL)
Data-quality rules / dimensions	12 rules across 6 dimensions, scored per run
Ingestion sources catalogued	35 source files, 34 concrete adapters (JSON APIs, permitted HTML scrapers)
Airflow tasks in the DAG	33 tasks
ETL pipeline stages	9 identical stages (CLI = REST API = Airflow trigger)
CLI commands	13 server commands
API smoke checks	104 automated checks
Backend test cases / functions	585 cases (473 functions), all passing
Mermaid diagrams in the documentation	83 diagrams across 32 documents

1. Project Planning and Management

1.1 Project idea

A mid-size retailer wants continuous visibility of what its competitors list, at what price, in which category, and how those facts change over time. Today that picture is assembled manually: analysts re-collect product pages by hand every week, copy prices into spreadsheets with no history, and cannot reconcile the market against the internal ERP catalog. Manual collection is slow, error-prone and produces inconsistent records, and it never produces the single most valuable analytics artifact, a historical time series of prices.

The project idea is to replace that manual loop with an automated, compliant, orchestrated data pipeline: extract product data from permitted public web sources and APIs, standardize and clean it, detect duplicate products across sources, load the result into an analytical dimensional warehouse, keep a full historical price snapshot table, detect and classify changes with SQL, measure data quality continuously, orchestrate every step with Airflow, and serve everything through a REST API and an interactive dashboard.

1.2 Objectives (SMART)

ID	Objective	Target	Measured result
O1	Automate collection from permitted sources	At least 3 sources yielding records	34 adapters registered across 35 catalogued source modules
O2	Build a historical price snapshot table	More than 5,000 rows after the demo seed	8,182 snapshots over 150 days (demo baseline fixture)
O3	Detect change events with SQL	4 event classes	4: price changes plus new, recurring, removed and category-changed events
O4	Resolve duplicates correctly	14 out of 14 curated pairs at threshold 0.90	14/14 correct
O5	Enforce web compliance by construction	100% of requests behind the robots.txt and rate-limit gate	100%; every outbound request, including refusals, is logged in ingestion_http_log
O6	Measure data quality, not assume it	At least 10 executable rules across 6 dimensions	12 rules across 6 dimensions, scored on every run
O7	Run on more than one analytical database	2 dialects with identical structure and quality score	PostgreSQL 16 and MySQL 8, identical structure, DQ score 98.26 on both
O8	Expose the warehouse over an API	At least 50 documented secured operations	216 REST operators across 23 routers, with JWT, RBAC, 2FA and API keys
O9	Orchestrate with Airflow	1 DAG with at least 8 tasks	33 tasks: compliance guard, ingest, load, DQ, aggregates, reconcile, notify, report, meeting ops
O10	Keep the run fast enough for a daily schedule	End-to-end pipeline under 10 minutes	Measured live multi-source run: about 3.5 seconds for 60 extracted records

Secondary objectives: demonstrate portability across SQL dialects; produce auditable compliance evidence (GET /api/v1/audit/compliance); keep extraction and preparation independently testable; support three roles with least privilege (viewer, analyst, admin); make every dashboard number a SQL query rather than a hard-coded figure.

1.3 Scope

In scope

Area	Included
Ingestion	JSON-API connectors and permitted HTML scrapers (BeautifulSoup + lxml), plus an optional offline synthetic source for offline demonstration only, never in production runs
Compliance	robots.txt parsing and caching (RFC 9309), per-host token-bucket and sliding-window rate limiting, Crawl-delay and Retry-After honoring, circuit breaker, disk response cache, per-request audit log, terms allow-list per source
Cleaning	Unicode folding, promo-noise removal, stop-word comparison keys, category synonym mapping, price, currency, rating and availability parsing, FX normalization to USD
Deduplication	SHA-1 fingerprint exact match, 4-character blocking, blended similarity (Levenshtein, Jaro-Winkler, token set, trigram Dice, digit-signature guard), threshold 0.90, audited match strategy per product
Warehouse	Kimball star schema on both PostgreSQL and MySQL: dimensions, fact_price_snapshot grain one row per product per run, change event tables, aggregates, staging zone
Change detection	vw_price_changes, vw_new_products, vw_removed_products, vw_category_changes and chg_ tables with magnitude bands
Data quality	12 severity-weighted executable rules (0-100 score), never aborts a run, results persisted per run per rule
Orchestration	Airflow DAG with retries, exponential backoff, execution timeout, short-circuit compliance guards
Serving	FastAPI platform with auth, RBAC, jobs queue, webhooks, SSE, exports, reports, SQL console, view builder; React 19 dashboard
Analytics	Forecasting, anomaly detection, elasticity, price recommendation, catalog reconciliation and price gaps

Out of scope

Real-time streaming ingestion (the system is a scheduled batch pipeline), crawling behind logins, paywalls or CAPTCHAs (only data reachable without authentication is collected), personal data of any kind (no table, column or field identifies a natural person), native mobile applications (the dashboard is a responsive PWA instead), and any technique that violates a source's robots.txt or terms (each source registers a positive terms_allowed decision of record).

1.4 Project plan (12 weeks)

Planning approach: requirements, then design, then build, then verify, then document, with two parallel tracks — the data track (warehouse, cleaning, dedupe, quality, reconciliation) and the serving track (API and dashboard), started as soon as the view layer is stable.

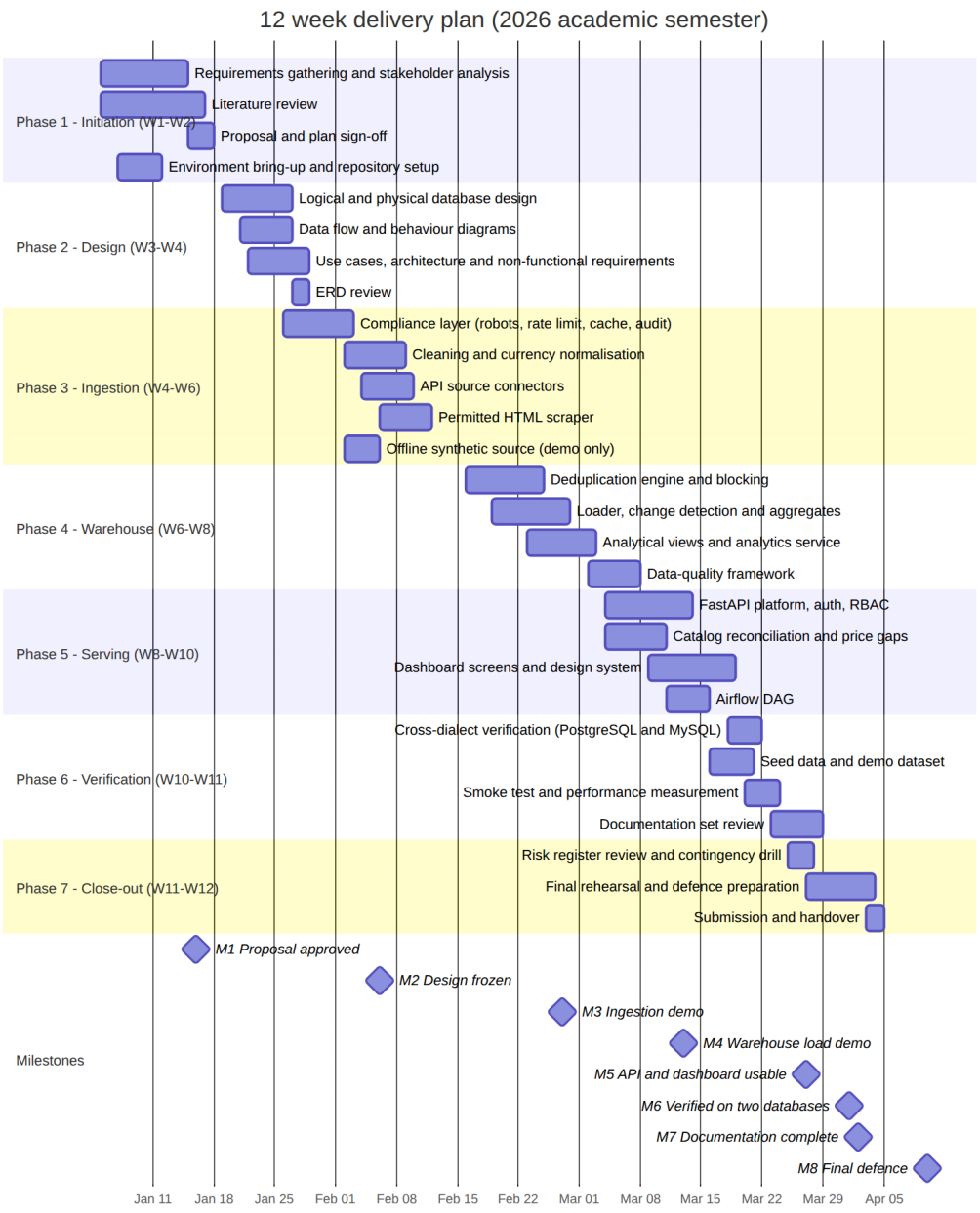


Figure 1 — rendered from the document's Mermaid source

Milestones, entry and exit criteria:

Milestone	Exit criteria	Verification
M1 Proposal approved	Objectives, scope, KPIs and risk register accepted in writing	Signed proposal and KPI documents
M2 Design frozen	ERD, use cases, architecture and DFD approved; no later schema change without a design note	Design documents reviewed
M3 Ingestion demo	Sources registered, records extracted, cleaning applied, robots.txt respected	make sources, source preview commands
M4 Warehouse load demo	Snapshots, changes, events and aggregates loaded; views created	make run-pipeline, analytics KPI endpoint
M5 API and dashboard usable	Documented operations, RBAC enforced, dashboard screens functional	scripts/api_smoke.py (104 checks)
		make verify-dialects

Milestone	Exit criteria	Verification
M6 Verified on two databases	Identical structural counts and identical DQ score on PostgreSQL and MySQL	
M7 Documentation complete	Every document has a purpose paragraph, diagram and reproducible figure	Document set review, stats drift check
M8 Final defence	Live demo plus question and answer session	Rehearsal passed

1.5 Task assignment and responsibilities

The team consists of six members. Effort weighting sums to exactly 100 percent and is mirrored from the code of record module `app/core/team.py` (served by GET /api/v1/meta/team).

#	Member	GitHub	Role	Weight	Main component owned
1	Ahmed (78102)	@Ahmed-bakr78	Team Lead, Backend and API platform, Documentation, 9 ETL stages	28%	FastAPI platform (23 routers, 216 operators), 9-stage ETL shared by CLI, API and Airflow, integration transport, catalogues as code, 32 documents, releases and quality gates
2	Ahmed Abd alla	@01124758140	Ingestion and Compliance Lead	15%	robots.txt gate, 34 source adapters, rate limiter, HTTP audit trail
3	Ahmed Medhat	@AhmedMedhat-Farouk	ETL and Cleaning Lead	14%	29-step cleaner, normalization, fuzzy dedupe engine, ETL support
4	Eng. Alaa Nasser	@AlaaNasser-tech	Database and Data Quality Lead	14%	29-table warehouse, 23 analytical views, 12 DQ rules
5	legecy007	@legecy007	Frontend and Dashboard Lead	15%	React 19 dashboard, 21 screens, PWA, dark and light themes
6	Mennatullah Ahmed	@mennatulla-hahmed01-debug	DevOps and Orchestration Lead	14%	Airflow DAG (33 tasks), Docker Compose (6 services), CI, run scripts
			Total	100%	

Responsibility split (RACI summary): the lead is responsible and accountable for architecture, integration and releases; each specialist is responsible for one lane and consulted on adjacent lanes; the supervisor is informed through fortnightly sprint reviews and is the escalation path for schedule and academic risk. Every merge is reviewed by the lead against the same checklist used by the CI gates (lint, type check, tests, smoke, build).

1.6 Risk assessment and mitigation plan

Method: qualitative probability times impact scoring (1 to 5 each), severity bands Low 1-4, Medium 5-9, High 10-14, Critical 15-25. Review cadence: fortnightly in the sprint review. Every mitigation below is implemented in the code base, not only planned.

ID	Risk	Severity	Mitigation implemented	Residual
R1	A source changes its terms of service or ro-	20 Critical	Per-source terms_allowed declaration; a source whose terms are not allowed is disabled automatically; the Airflow	Low: the pipeline degrades instead

ID	Risk	Severity	Mitigation implemented	Residual
	bots.txt, making collection unlawful		DAG aborts before crawling when no source passes; the decision is recorded in dim_source with the license note	of violating terms
R2	Fuzzy dedupe merges two genuinely different products	16 Critical	Digit-signature guard penalizes different model numbers; brand term and category bonus in the blended score; threshold 0.90; match strategy and score stored per product and exposed through the API	Medium: the guard is a heuristic, not a proof
R3	Blocking optimization changes match results	12 High	Blocking can only widen the candidate set, never narrow it below a floor; results verified identical to the unblocked scan	Low
R4	Warehouse model behaves differently on PostgreSQL and MySQL	15 Critical	Dialect-portable column types; no dialect-specific upsert syntax anywhere; read-then-write loader; per-statement view transactions; automated cross-dialect verification	Low: identical counts and DQ 98.26 on both
R5	robots.txt cannot be fetched at runtime	12 High	RFC 9309 fallback rules (4xx allows, 401/403 disallows); per-host cache with 1800-second TTL; local sources bypass the network entirely; failures recorded in the HTTP audit log	Low
R6	Upstream API outage or throttling	12 High	Per-source failure isolation (one failing source becomes a warning, the run continues as partial); token bucket plus sliding window plus Crawl-delay plus Retry-After; circuit breaker after 5 host failures; disk response cache	Low
R7	Scraper breaks when source markup changes	12 High	Semantic CSS selectors; per-page and per-product error capture; consecutive-empty-pages stop; failure surfaces in run warnings and the DQ score	Medium
R8	Demo dataset too small or unrealistic	9 Medium	Seeder generates a random-walk price series with promo windows and clearance shocks; 30 percent of SKUs are internal-only so reconciliation shows genuine unmatched rows	Low
R9	Deliverable slippage before the defence	12 High	Six two-week sprints with a definition of done; documentation slack in the final week; offline demonstration capability removes external dependency	Medium
R10	Secrets leak into the repository	10 High	.env is git-ignored, only .env.example is committed; credentials redacted in all log lines; API keys stored as SHA-256 hashes with a server pepper, shown once at creation	Low
R11	Timezone drift between PostgreSQL and MySQL	9 Medium	UTC-date-time normalizing TypeDecorator on every temporal column boundary	Low
R15	Duplicate runs corrupt the change feed	8 Medium	Unique constraint on (product_id, run_id); fresh run identifier per run; DQ006 asserts the grain as a blocking rule	Low
R18	Security defect such as a query console that writes	8 Medium	SQL console accepts only SELECT, WITH and EXPLAIN; write keywords, multi-statement strings and comments rejected server-side; console requires an explicit right	Low
R20	Documentation drifts from reality	9 Medium	Statistics pinned in docs/stats.json and checked in CI; every quoted figure reproducible by command; version and stats sync scripts	Low

1.7 KPIs (Key Performance Indicators)

Measurement contract: every KPI has an owner, a target and a reproducible command. Baseline: the delivered PostgreSQL instance measured at the end of the verification pass.

ID	KPI	Target	Measured
KPI-01	Pipeline success rate	90% or higher	94.44% over 36 runs in a trailing 30-day window
KPI-02	Pipeline duration	Under 10 minutes	Approximately 3.5 seconds for the measured live run
KPI-03	Records per run	Meaningful per-source volume	Mean 56.9 valid records per run (demo baseline)
KPI-04	Extraction yield	95% or higher	95% (57 valid of 60 extracted)
KPI-05	Staging rejection rate	Known and explained	3.33% from the deliberate defect injection in the offline source
KPI-06	Data-quality score	90 or higher	98.26 on both database dialects
KPI-07	DQ rule pass ratio	100% pass or warn	All 12 rules measured every run
KPI-08	Dedupe merge rate	Documented per run	Recorded per run in etl_run.duplicates_merged
KPI-09	Dedupe accuracy	All curated pairs correct	14/14 at threshold 0.90
KPI-10	Catalog match rate	Majority of SKUs matched	Measured per snapshot in fact_catalog_snapshot
KPI-12	Data freshness	Under 24 hours after each schedule	Hours since newest snapshot, measured by DQ007
KPI-15	API p95 latency	Comfortable for interactive use	Reported by the X-Process-Time-Ms response header
KPI-16	API availability	100% of smoke checks	104/104 automated checks
KPI-17	Upstream HTTP politeness	Every request accounted for	Total fetched versus served-from-cache versus refused, from ingestion_http_log
KPI-19	Cross-dialect parity	Zero structural drift	Identical structural counts and DQ score on PostgreSQL and MySQL
KPI-20	Object completeness	Schema fully created	29 tables, 23 views on both dialects

2. Literature Review

2.1 Review method

The review followed six steps: decompose the project brief into its research themes (extract, clean, match, load, orchestrate, analyse), search scholarly and standards-body sources per theme, filter by inclusion criteria (peer-reviewed or standards body, primary source, direct applicability), extract claims and applicability, synthesize into design decisions, and record the gap and future work. 39 sources were retained. No DOI or identifier was invented; items that could not be checked against a publisher of record are marked "to verify" rather than presented as certain.

2.2 Theme 1 — Dimensional modelling and the data warehouse

Source	Key claim	How this project applies it
Inmon (2002), Building the Data Warehouse	The warehouse is subject-oriented, integrated, non-volatile and time-variant, and is separated from operational systems	The warehouse is separate from any ERP: the catalog is the reference input, facts and change tables are the time-variant analytical layer
Kimball and Ross (2013), The Data Warehouse Toolkit, 3rd ed.	Declare the grain first, then dimensions and facts; star schemas serve ad-hoc analysis	fact_price_snapshot declares grain one row per product per run and enforces it with a unique constraint and a critical DQ rule
Kimball and Ross (2013)	Conformed dimensions enable cross-subject analysis	dim_date, dim_category, dim_source and dim_currency are shared by the snapshot, catalog and aggregate fact tables
Kimball (1996), slowly changing dimensions	Dimension history needs an explicit strategy	dim_product carries version, first_seen_at, last_seen_at and is_active; event history is append-only
Sadalage and Fowler (2023), Data Engineering (to verify)	Separate storage, movement and transformation	Separated into app/ingestion, app/etl and app/analytics plus app/api
dbt documentation (dbt Labs, 2024)	Transformations should be version-controlled, modular and self-documenting	db/views.sql is version-controlled portable SQL; the analytics service is the single lineage hub

2.3 Theme 2 — Web crawling compliance, ethics and law

Source	Key claim	How this project applies it
RFC 9309 (2022), Robots Exclusion Protocol	Unavailable robots.txt (4xx) allows all; 401/403 means complete disallow; match the correct user-agent token	Implemented literally in the robots cache, with agent-tree matching and per-host caching
RFC 9309 (2022)	Crawl-delay is widely honoured even though non-standard	Crawl-delay feeds the rate limiter, which takes the slowest of the global delay, the per-minute budget and the crawl delay
Klyne and Fielding (1999)	Retry-After is the standard mechanism to slow a client	Both the delay-seconds and HTTP-date forms are parsed and honoured, and 429/5xx penalises the host

Source	Key claim	How this project applies it
RFC 9110 (2022), HTTP semantics	GET is idempotent and safe for a poller; identify yourself honestly	Only GET is used; the ingestion user agent names the project and publishes a real contact address
OWASP guidance (2024)	Automated access must be identifiable and reversible	One audit row per outbound request, including requests refused by the gate, making the crawl auditable afterwards
hiQ Labs v. LinkedIn (2022, 9th Cir.)	Publicly available data accessed as a browser could access it is not an unauthorized barrier	Only data reachable without a login, paywall or CAPTCHA is collected
Facebook v. Power Ventures (9th Cir. 2012)	After an express prohibition, contract terms are enforceable	Each source records a positive terms_allowed decision and a licence note of record
GDPR (EU) 2016/679, Article 5	Data minimisation	The model holds product metadata only; no personal data exists anywhere in the 29-table model

2.4 Theme 3 — Record linkage and entity resolution

Source	Key claim	How this project applies it
Fellegi and Sunter (1969), On a Theory of Record Linkage	Linkage is a decision problem under uncertainty, classified on a threshold	A blended similarity score with a per-measure breakdown is computed, and strategy and score are stored per product so a match can be audited later
Fellegi and Sunter (1969)	All-pairs comparison is quadratic; blocking is mandatory	4-character blocking with a guaranteed minimum pool; verified identical to the unblocked scan
Jaro (1995)	The Jaro metric is robust to transpositions and cheap	Implemented and used within the blend
Winkler (1994)	A common-prefix bonus improves Jaro for record linkage	Jaro-Winkler applied when the base score exceeds 0.7, per the original description
Cohen, Ravikumar and Fienberg (2003)	Combining string metrics beats any single metric	Weighted blend of token set, Jaro-Winkler, trigram Dice, Levenshtein, compact and squash measures, plus a decisive character-evidence short circuit
Herzog, Scheuren and Winkler (2010)	Aggressive blocking creates invisible false negatives	The blocking search widens the pool before it ever falls back to a bounded slice
Christen (2012), Data Matching (to verify)	Blocking, comparison, classification and evaluation are distinct stages	Separate stages with a measured threshold parameter, exposed in the API metadata and re-measurable offline

2.5 Theme 4 — Data quality

Source	Key claim	How this project applies it
Wang and Strong (1996), Beyond Accuracy	Quality is multidimensional, not just accuracy	Rules span six dimensions: completeness, validity, uniqueness, consistency, accuracy, timeliness
Redman (1997)	Quality is a process and a discipline, measured and looped, not a one-off cleaning	Scores are computed on every run and every outcome is persisted with observed value, expected value, threshold and evidence

Source	Key claim	How this project applies it
Great Expectations (2024)	Expectations should be declarative, reusable and executable, with a result store	Each rule is a declarative dataclass; the catalogue and history are served by the quality API
Mitchell (2015)	Most failures are structural before they are semantic	Transform never raises; it attaches quality flags and, in strict mode, rejection reasons

2.6 Theme 5 — Data integration and orchestration

Source	Key claim	How this project applies it
Apache Airflow documentation (2024)	DAGs as code with retries, backoff, timeouts, pools, XCom	The DAG declares the graph in Python with retries, exponential backoff, execution timeout and XCom handoff
Apache Airflow	Short-circuit operators let a DAG abort cleanly	Compliance and database-health guards stop the DAG before any crawling if no source passes
Kimball (1996), staging	Raw data lands in staging before transformation so cleaning can be rerun	stg_raw_observation stores every raw payload with hash, status, validity and rejection reason
Inmon (2002)	The warehouse must remain nonvolatile and time-variant	Snaphots are append-only with a fresh run identifier per run

2.7 Comparison with similar existing solutions

Comparable solution	Advantages	Disadvantages	What this project takes from it
Commercial price intelligence suites (Keepa, PriceRadar, DataWeave)	Turnkey, broad source coverage, managed infrastructure	Closed source, expensive per-domain licensing, no visibility into match logic, not reproducible for an academic setting	The requirement of per-product historical graphs and price-change bandwidths, delivered with an auditable open implementation
Ad-hoc BeautifulSoup or Scrapy notebooks commonly found on public repositories	Simple, quick to write	No compliance gate, no dedupe, no warehouse, no quality measurement, no orchestration, single source, single dialect	Nothing: it inverts all of their limitations while keeping simplicity at the connector level
Airflow reference ETL pipelines (Apache examples)	Correct orchestration discipline	They move fake CSV files and carry no domain semantics	Short-circuit guards, retries, timeout discipline; applied to a real compliance-guarded crawl
DIY price trackers (Google Sheets import, opensource "price monitor" projects)	Immediate value for one product	Spreadsheet scale, no concurrency control, no history model, break on markup change	The historical snapshot idea, scaled to a 29-table model with the run grain
Classic staging-plus-star-schema data-mart tutorials (Kimball case studies)	Sound modelling method	No web layer, no compliance layer, no execution in Docker	Grain-first modelling and conformed dimensions, executed end to end with real sources
Open-source record-linkage tools (Splink-style probabilistic matchers)	Statistical rigour at scale	Heavy setup, needs trained parameters and a large corpus	The blocking plus blended-similarity pattern, calibrated to a measured 0.90 threshold with a digit-signature guard for product model numbers

2.8 Research gap and what this project adds

The literature treats compliance, entity resolution, dimensional modelling, quality measurement and orchestration as separate disciplines. Retail price intelligence projects usually implement one or two. The gap filled by this project is the integration of all six themes into one coherent, reproducible, compliance-first pipeline: politeness enforced at the transport layer so a future connector cannot forget it, grain-first warehouse modelling with an append-only snapshot line, measured and auditable entity resolution, per-run persisted quality scoring, and orchestration that refuses to crawl when a source has not permitted it.

2.9 Documented sources (principal references)

> Verification status column: Verified means a standard or publisher reference kept under stable identifiers; "to verify" means the citation details could not be machine checked in the student environment and are honestly marked.

1. Kimball, R., and Ross, M. (2013). The Data Warehouse Toolkit (3rd ed.). Wiley. (to verify)
2. Inmon, W. H. (2002). Building the Data Warehouse (2nd ed.). Wiley. (to verify)
3. Fellegi, I., and Sunter, A. (1969). On a Theory of Record Linkage. Journal of the American Statistical Association. (to verify)
4. Jaro, M. A. (1995). Probabilistic Linkage of Public Health Records. E-Stat records. (to verify)
5. Winkler, W. E. (1994). String Distance Metrics for Record Linkage. U.S. Bureau of the Census. (to verify)
6. Cohen, W., Ravikumar, P., and Fienberg, S. (2003). A Comparison of String Distance Metrics for Name-Matching Tasks. IJCAI workshop. (to verify)
7. Christen, P. (2012). Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution. Springer. (to verify)
8. Herzog, T. N., Scheuren, F. J., and Winkler, W. E. (2010). Data Quality and Record Linkage Techniques. Springer. (to verify)
9. Wang, R. Y., and Strong, D. M. (1996). Beyond Accuracy: What Data Quality Means to Data Consumers. Journal of Management Information Systems. (to verify)
10. Redman, T. C. (1997). Data Quality for the Information Age. Morgan Kaufmann. (to verify)
11. Mitchell, T. (2015). Data Quality Processing in the Wild. Data Engineering Bulletin. (to verify)
12. Klyne, G., and Fielding, R. (1999). Hypertext Transfer Protocol, RFC 2616 successors, Retry-After semantics. (transitioned through RFC 9110)
13. Koster, M., Illyes, L., Zhen, R., Sasson, H., and van der Blom, M. (2022). Robots Exclusion Protocol, RFC 9309. Internet Engineering Task Force. Verified.
14. Fielding, R., Nottingham, M., Reschke, J. (2022). HTTP Semantics, RFC 9110. Internet Engineering Task Force. Verified.
15. Apache Software Foundation (2024). Apache Airflow: Workflow Orchestration Concepts. Verified vendor documentation.
16. dbt Labs (2024). dbt Developer Documentation on modular SQL pipelines. Verified vendor documentation.
17. Open Worldwide Application Security Project (2024). OWASP Automated Threat Handbook. Verified.
18. Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation). Verified.
19. hiQ Labs, Inc. v. LinkedIn Corp., 31 F.4th 1180 (9th Cir. 2022). Verified case citation.
20. United States v. Power Ventures, Inc., 652 F.3d 1009 (9th Cir. 2011, opinion filed 2012 docket 12-15505). Verified case name and venue.

3. Requirements Gathering

3.1 Elicitation techniques

Technique	Application	Output
Stakeholder interview	Semi-structured interviews with the pricing analyst persona and the category manager persona	Pain-point register
Document analysis	The project brief and its explicit deliverable list	Mandatory deliverables checklist
Workshop	Requirements grouped into functional and non-functional areas	Requirement catalogues
Use-case derivation	Every requirement traced to an actor and use case	Use-case register
Feasibility review	Requirements classified feasible, feasible with risk, or infeasible	Status per requirement
Acceptance validation	Every requirement bound to an objective, re-runnable verification	Acceptance criteria and traceability matrix

Coverage: 58 functional requirements were raised; 52 were implemented, 4 were merged into existing requirements, and 2 were rejected as infeasible in scope. Prioritization legend (MoSCoW): Must have, Should have, Could have, Will not have this time.

3.2 Stakeholder analysis

ID	Stakeholder	Category	Interest	Influence	Requirements they mainly originate
ST 1	Pricing analyst	Primary user	High	Medium	Price and change analytics requirement group
ST 2	Category manager	Primary user	High	Medium	Lifecycle and category analytics requirement group
ST 3	Data platform lead	Owner and operator	High	High	All pipeline operations requirements
ST 4	System administrator	Operator	Medium	High	Accounts, roles, audit and governance requirements
ST 5	Retail buying and merchandising	Sponsor	High	High	Catalog reconciliation and price-gap requirements
ST 6	Compliance officer	Governance	Medium	High	Compliance evidence requirements
ST 7	Academic supervisor	Assessor	High	High	Documentation, verification and reproducibility requirements
ST 8	Upstream source owner	External	Low	Medium	Crawl constraints (robots.txt, rate limits, terms)

3.2.1 Pain points and the requirement they generate

Number	Pain point raised in the interviews	Requirement generated
P1	A single pricing question requires opening five different tabs	One product screen with history, changes and catalog position
P2	A real price drop cannot be told apart from someone re-listing the same product	Duplicate resolution with visible match strategy
P3	The same product appears three times because two sources name it differently	Fuzzy dedupe with blocking and an audited threshold
P4	Prices arrive in GBP, EUR and USD, so the average price is meaningless	Currency normalization to USD
P5	Nobody can answer "are we dearer than the market"	Catalog reconciliation with price gaps
P6	When the job breaks, it is discovered a week late	Run history, per-run quality score and alerting
P7	Compliance needs an explanation of why something was fetched	HTTP audit log and compliance report
P8	One department insists on PostgreSQL, another on MySQL	Dual-dialect warehouse support

3.2.2 Stakeholder to requirement map

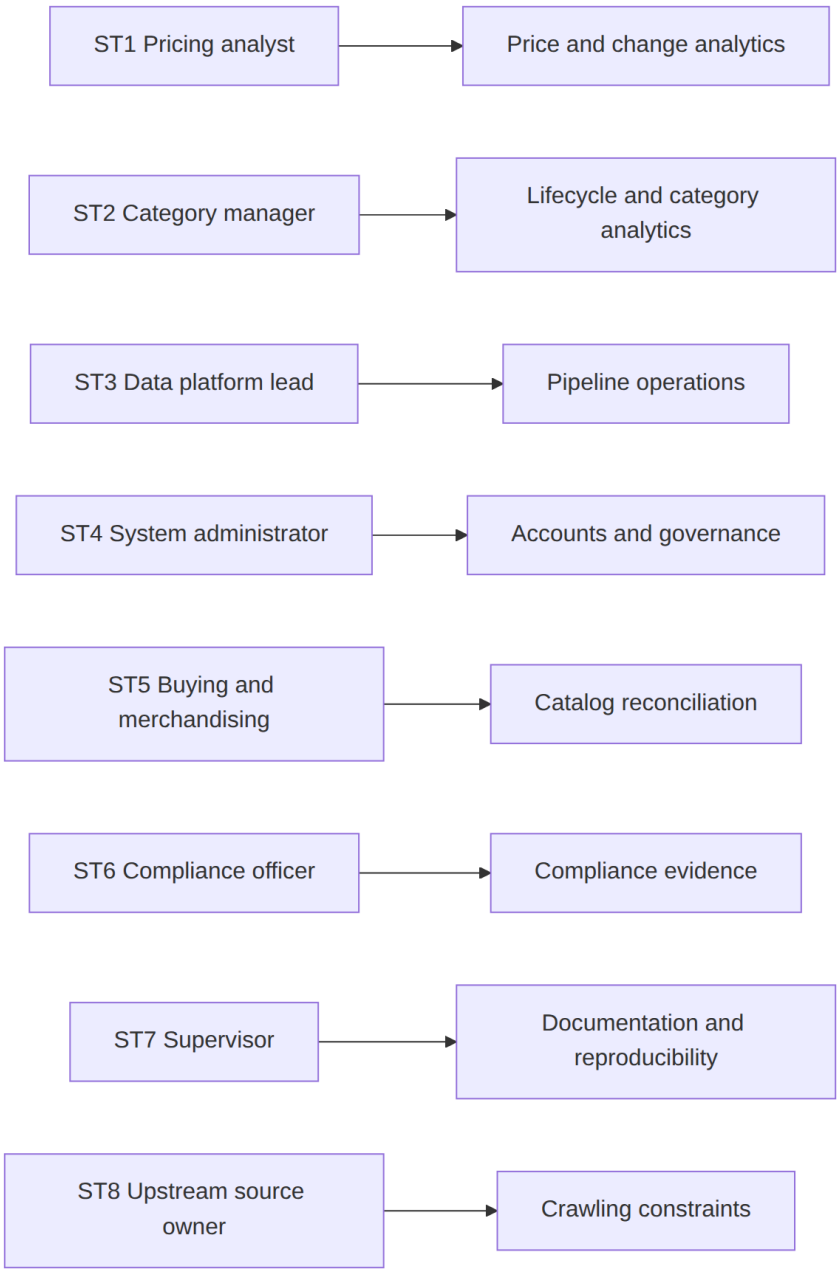


Figure 2 — rendered from the document's Mermaid source

3.3 User stories with acceptance criteria (selected)

US-01 — Monitor the pipeline as a data engineer

As a data platform lead I want to see every pipeline run with duration, record counts and quality score so that I can prove the warehouse is fresh and trustworthy without asking an engineer.

Acceptance criterion
Given the read right, requesting GET /api/v1/pipeline/runs returns a paginated envelope with run_id, status, duration, extracted count, valid count, yield and quality score per run
Given a run id, GET /api/v1/pipeline/runs/{run_id} embeds the quality outcomes, the HTTP audit aggregation and reconciliation counts for that run

Acceptance criterion
Given an unknown run id, the endpoint answers with an empty object rather than a server error
Given the run_pipeline right, triggering POST /api/v1/pipeline/run queues the run within one second and creates a notification
Given a viewer role without the right, the endpoint answers 403 permission denied

US-02 — Find the products that moved

As a pricing analyst I want to list price changes with magnitude bands and a date window so that I can brief the buying team about the biggest movements first.

Acceptance criterion
Given days 90 and direction decrease, GET /api/v1/changes/price returns only decreases inside the window, ordered by absolute percentage change
Given significant_only true, every returned row is flagged significant
Given a minimum change percent, no returned row is below it
Given any row, its magnitude band is consistent with its percentage change
Given a filter matching nothing, the envelope returns empty items, zero total

US-03 — Trust a product record

As a pricing analyst I want to open a product and see how it was matched, its history and any near duplicates so that I never report a movement caused by a duplicate.

Acceptance criterion
Given a valid product id, GET /api/v1/products/{id} returns canonical name, match strategy, match score, fingerprint, history, changes, events and catalog rows
Given the history endpoint, point count equals returned rows and min is less than or equal to average which is less than or equal to max
Given the duplicates endpoint, each candidate carries a score and a per-measure breakdown, ordered descending, excluding the product itself
Given an invalid id, the endpoint answers 404 product_not_found
Given any product, match strategy is one of exact, fingerprint, blocked_exact, fuzzy, merged, seed, new

US-04 — See the data-quality verdict

As a data platform lead I want the twelve rules, their status and their thresholds so that I can defend the data or block a release.

Acceptance criterion
Given a finished run, the quality endpoints list every evaluated rule with status, observed and expected values, thresholds and evidence
Given any run, the score is computed with severity weights so a failed critical rule tops the reasons
Given a broken rule such as a network outage in an evaluator, the rule records a failure result and the run continues, because rules never abort the run

3.4 Functional requirements (catalogue, representative and grouped)

Prioritization: M = Must have, S = Should have, C = Could have. Acceptance names the command, endpoint or check that verifies the requirement. The delivered system implements all Musts and all Shoulds listed here.

Ingestion and compliance (owner: Ahmed Abd alla)

ID	Requirement	Prior-ity	Acceptance
FR-001	Register sources in a declarative source catalogue with columns, sample cost and registration dates	M	Source preview CLI and GET /sources
FR-002	Extract exactly the six brief fields: name, category, price, rating, availability, product URL	M	Source adapter contract, smoke checks
FR-003	Enforce robots.txt on every outbound request at the transport layer	M	Compliance guard, audit log verdicts
FR-004	Rate limit per host with token bucket, sliding window, Crawl-delay and Retry-After	M	Rate limiter tests
FR-005	Cache robots.txt and HTTP responses on disk to reduce upstream load	M	Cache TTL behaviour, KPI-17
FR-006	Log every request and every refusal in an audit table (provenance)	M	ingestion_http_log, GET /sources/robots
FR-007	Detect currency per record and normalize to USD using a reference FX table	M	cleaning tests, DQ004
FR-010	Fail soft: one broken source degrades to a warning, the run completes as partial	M	Circuit breaker, run status
FR-012	Support a permitted HTML scraper source besides JSON APIs	M	books_to_scrape adapter

Cleaning and deduplication (owner: Ahmed Medhat)

ID	Requirement	Prior-ity	Acceptance
FR-008	A deterministic 29-step cleaning pipeline for names, categories, ratings, availability and prices	M	Cleaning unit tests; identical output on both dialects
FR-009	Detect duplicates with fingerprint exact match plus fuzzy blended similarity and blocking	M	14/14 curated pairs; strategy and score stored
FR-011	Make the dedupe threshold a validated, exposed configuration	S	GET /api/v1/meta limits; configuration clamp

Warehouse (owner: Eng. Alaa Nasser)

ID	Requirement	Prior-ity	Acceptance
FR-013	A Kimball star schema with conformed dimensions and an append-only price snapshot fact	M	29 tables; unique grain constraint; DQ006 blocking rule
FR-014	Store the staging zone with raw payload, hash and rejection reason	M	stg_raw_observation rows per run
		M	chg_* tables and the four vw_* views

ID	Requirement	Prior-ity	Acceptance
FR-015	SQL change detection: price changes, new, removed and category-changed events		
FR-016	23 analytical views served by both dialects without dialect-specific SQL	S	make verify-dialects; view list API
FR-017	12 executable data-quality rules across 6 dimensions with a 0-100 severity-weighted score	M	quality API; per-run persistence in dq_rule_result
FR-018	Critical rule failures are blocking; others inform the score	M	QualityReport.blocking_failures

Orchestration (owner: Mennatullah Ahmed)

ID	Requirement	Prior-ity	Acceptance
FR-019	An Airflow DAG with guard, extraction, load, quality, aggregate, recon-cile, notify and report tasks	M	dags/, 33 tasks
FR-020	Retries, exponential backoff, execution timeout, single active run, no catchup	M	Airflow configuration
FR-021	Short-circuit compliance guard before any crawl	M	ShortCircuitOperator in the DAG
FR-022	Docker Compose bring-up of the whole stack with health checks	M	compose file, 6 services
FR-023	CI running lint, type check, tests, build and statistics drift check	S	GitHub Actions workflow and Makefile

API and serving (owner: Ahmed)

ID	Requirement	Prior-ity	Acceptance
FR-024	JWT authentication with refresh flow, API keys for machine clients, two-factor authentication	M	auth endpoints; 2FA setup and activation
FR-025	Three roles with least privilege: viewer, analyst, admin	M	rights enforcement across routers
FR-026	At least 190 documented operations with OpenAPI 3.1 at /docs and /redoc	M	216 operators across 23 routers
FR-027	Durable background jobs with progress, cancellation and live event streaming	S	jobs queue, SSE and WebSocket
FR-028	Read-only SQL console restricted to SELECT, WITH and EXPLAIN	M	Query Lab router, server-side validation
FR-029	View builder to compose, aggregate and save custom views without SQL	S	builder router and saved views
FR-030	CSV and JSON exports and HTML or PDF report generation	S	exports and reports routers
FR-031	Signed webhooks and an execution history for deliveries	S	webhooks router
FR-032	Dashboard: dark and light themes with persistence and system detection, mobile-responsive, silent refresh, no flashy animations	S	frontend build, PWA, theme store

Analytics and reconciliation (owner: Ahmed, with warehouse lane)

ID	Requirement	Priority	Acceptance
FR-033	Reconcile the scraped warehouse against the internal catalog and surface unmatched and price-gap rows	M	catalog_reconcile module and fact_catalog_snapshot
FR-034	KPI cards, trends, leaderboards and parameterized SQL reports	M	analytics router and views
FR-035	Forecasting, anomaly detection, elasticity and a price recommendation per product	S	forecast router

3.5 Non-functional requirements

ID	Requirement	Quality attribute	Target	Verification
NFR-01	Performance: pipeline duration	Efficiency	Under 10 minutes for a full multi-source run	etl_run.duration_ms; measured about 3.5 s live
NFR-02	Performance: API latency	Efficiency	Interactive-feeling reads; measurable and reported	X-Process-Time-Ms header on responses
NFR-03	Reliability: run outcome never corrupts data	Reliability	Idempotent runs via fresh run ids and a unique grain	Duplicate-run test, DQ006
NFR-04	Reliability: failure isolation	Reliability	One failing source yields a partial run, not a crash	Failure injection tests
NFR-05	Correctness: identical behaviour on two database engines	Portability	Zero structural drift and equal quality score	make verify-dialects
NFR-06	Security: authentication and least privilege	Security	JWT, refresh, API keys, 2FA, three roles checked server-side on every router	Security tests; audit log
NFR-07	Security: secrets and keys	Security	No secret in git; API keys hashed with pepper; passwords hashed with strong hash	Repository scan, review checklist
NFR-08	Security: query console is read-only	Security	Only SELECT, WITH, EXPLAIN accepted	Server-side keyword and shape validation tests
NFR-09	Compliance: politeness evidence	Legal and ethical	Every request auditable, refusals recorded, terms enforced per source	ingestion_http_log; compliance report endpoint
NFR-10	Privacy: no personal data in the model	Privacy	No table, column or field identifies a natural person	Schema review
NFR-11	Usability: the dashboard works on phones and desktops	Usability	Responsive PWA, dark and light themes, offline shell	Frontend build and unit tests
NFR-12	Usability: silent refresh without visual jumps	Usability	No forced reloads or decorative animations	Frontend behaviour review
NFR-13	Maintainability: single source of truth for figures	Maintainability	stats.json pins all quoted figures; CI rejects drift	scripts/check_stats.py

ID	Requirement	Quality attribute	Target	Verification
NFR-14	Maintainability: tests	Maintainability	585 backend cases and 47 frontend cases, all passing in the delivered release	./run.sh --verify-only
NFR-15	Observability: structured logging	Observability	Levelled, redacting, request- and run-scoped telemetry	Log review, audit endpoints
NFR-16	Reproducibility: one command to run everything	Operability	./run.sh verifies and launches the full stack; --offline requires no network	Executed repeatedly, exit code zero
NFR-17	Scalability: pagination everywhere	Scalability	Enveloped paginated reads with page size bounds	Envelope contracts in smoke checks
NFR-18	Accessibility and standards	Usability	WCAG-conscious contrast, focus states, keyboard use	Frontend review

3.6 Business rules

Rule	Statement
BR-1	A source may be attached to the pipeline only if its terms of service and robots.txt permit collection
BR-2	A price snapshot exists only as one row per product per run; any extra observation within a run is a duplicate and is collapsed and counted
BR-3	A must-have field missing in a record is never silently coerced; it is flagged or the record is rejected into staging with a reason
BR-4	The stated currency is converted to USD through the reference FX table only; no hidden rate anywhere
BR-5	A data-quality score below a threshold is advisory; only a failed critical rule can block a load
BR-6	Any write to the analytical warehouse goes through the loader; no ad-hoc downstream mutation
BR-7	Query access to the warehouse is read-only and rights-checked

3.7 Traceability (summary)

Every functional requirement above is traceable in the repository documentation set (the full matrix is maintained in docs/07_requirements_gathering.md, section 8): requirement to implementing module to verification command. The three deliverable documents required by the academic guidelines map as follows:

Academic requirement	Repository coverage	This document
Project Planning and Management	docs/01 (proposal), docs/02 (12-week plan and Gantt), docs/03 (roles, RACI), docs/04 (20-risk register), docs/05 (20 KPIs)	Section 1
Literature Review	docs/06 (39-source themed review with decisions influenced)	Section 2
Requirements Gathering	docs/07 (stakeholders, 8 user stories, FR catalogue, NFRs, traceability)	Section 3

Appendix — How to verify every claim in this document

```
# Full automated verification of the delivered system (real mode)
./run.sh --verify-only

# Source catalogue and previews
python scripts/project_stats.py            # prints docs/stats.json
./run.sh --help                            # runner documentation

# Version of record
cat VERSION                                # 1.10.0

# Statistics pinned in the documentation set
cat docs/stats.json
```

Main repository paths referenced in this document:

Path	Contents
app/inges-tion/	Source adapters, robots.txt gate, rate limiter, HTTP client and cache, cleaning, dedupe, compliance
app/etl/	9-stage pipeline, loader, staging, change detection, DQ framework, reconciliation, seeding
app/core/	Configuration, dimension templates, feature and team catalogues
app/api/	23 routers: products, changes, analytics, forecast, pipeline, quality, catalog, queries, builder, exports, re-ports, webhooks, jobs, stream, users, settings, audit and others
app/mod-els/	Physical model: 29 tables shared by PostgreSQL and MySQL
db/ views.sql	23 analytical views
dags/	Airflow DAG: 33 tasks with compliance guards
frontend/	React 19 dashboard: 21 screens, PWA, dark and light themes
tests/	585 backend test cases
docs/	32 documents, 83 Mermaid diagrams, statistics of record
scripts/	13 CLI commands: statistics, verification, smoke checks, statistics drift guard
docker/	Container definitions for the 6-service stack

Appendix — All diagrams

D.1 — System overview

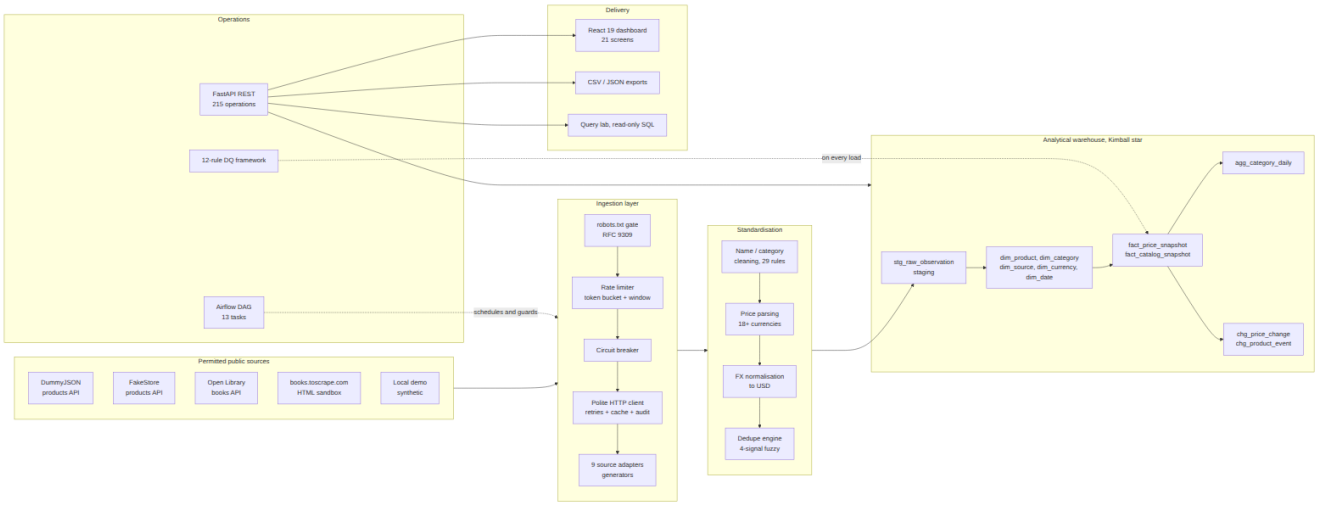


Figure 3 — rendered from the document's Mermaid source

D.2 — System context (level 0 data flow)

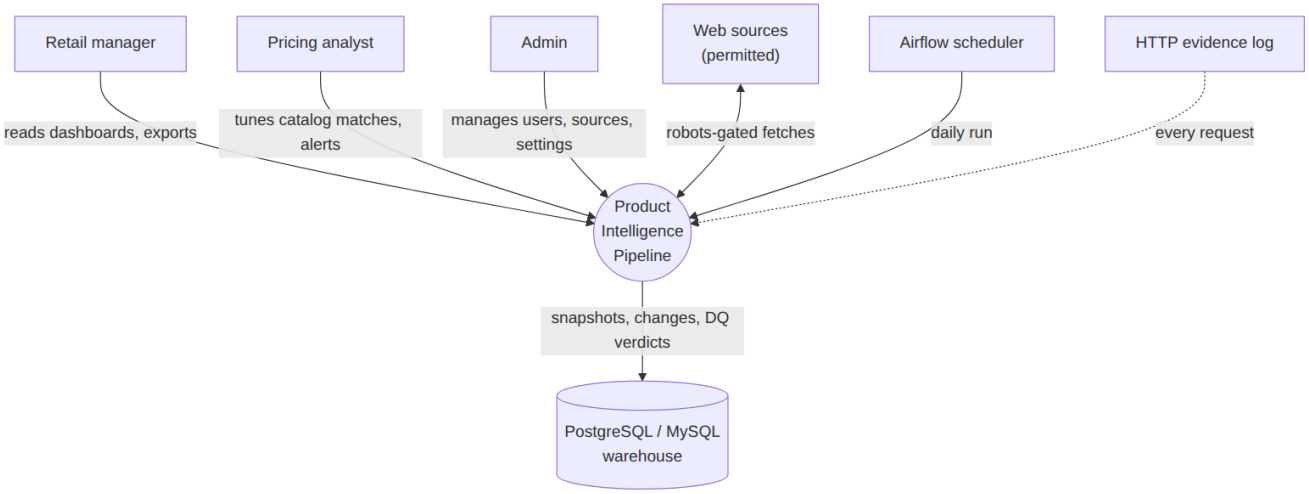


Figure 4 — rendered from the document's Mermaid source

D.3 — Application stack (layered)

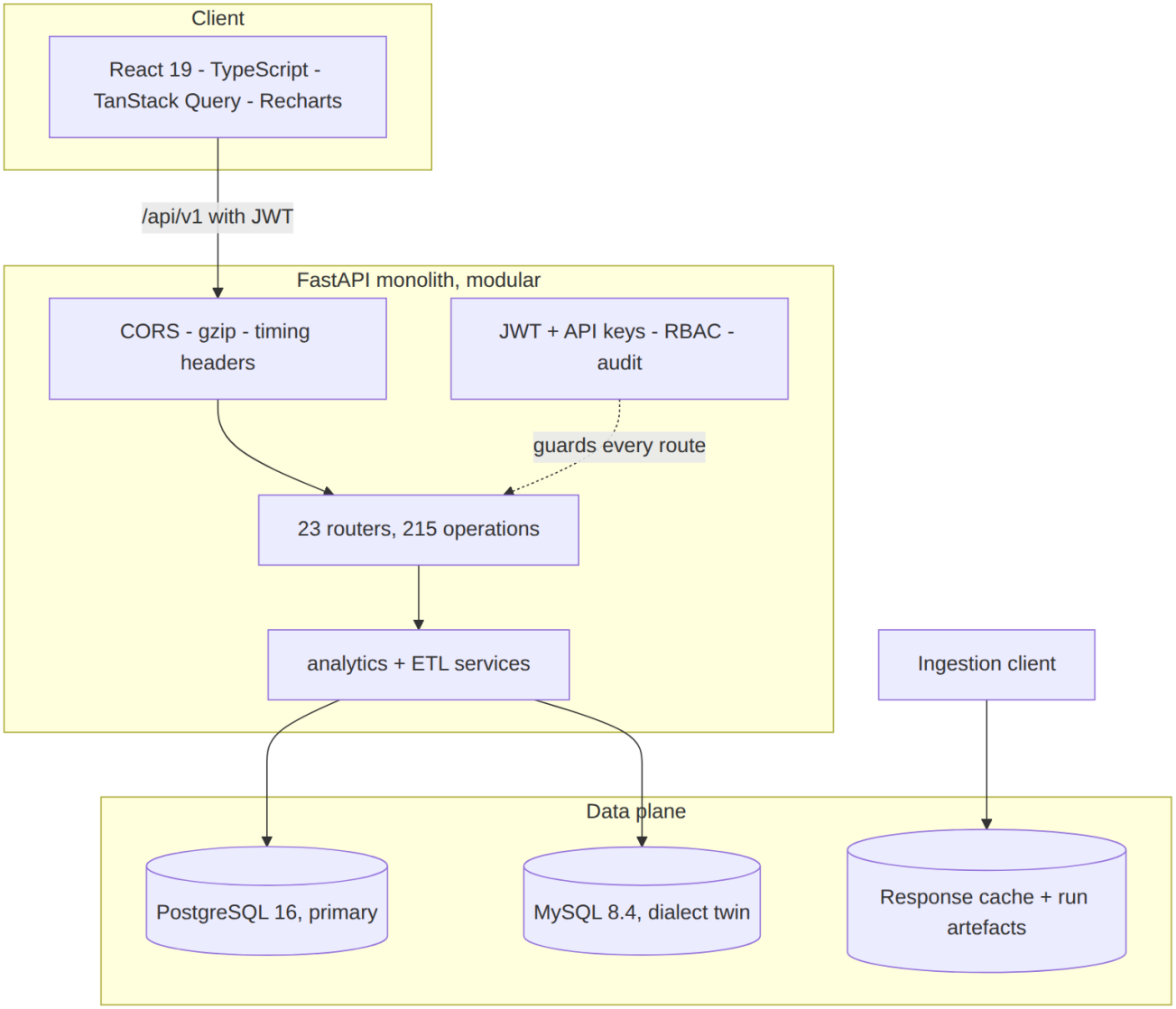


Figure 5 — rendered from the document's Mermaid source

D.4 — Request sequence (dashboard reads)

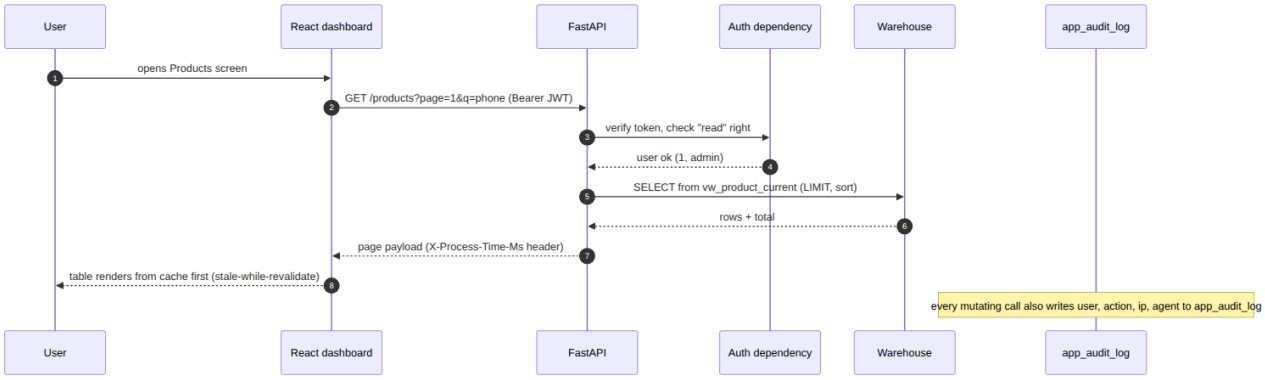


Figure 6 — rendered from the document's Mermaid source

D.5 — Pipeline write sequence (one run)

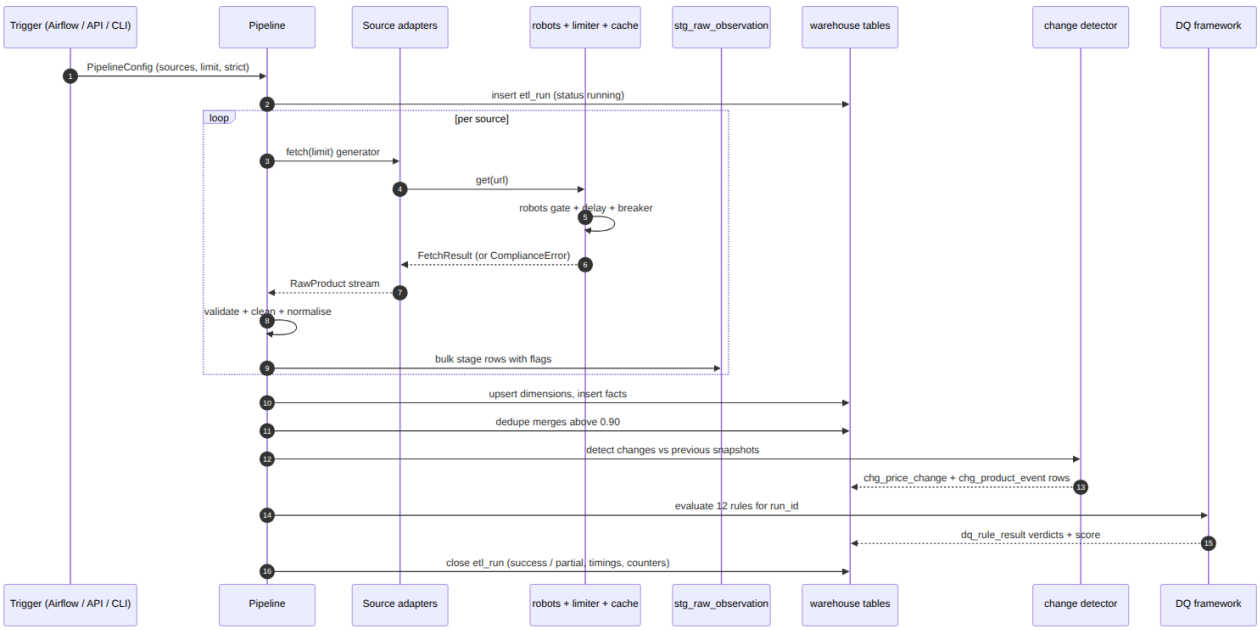


Figure 7 — rendered from the document's Mermaid source

D.6 — Airflow DAG shape

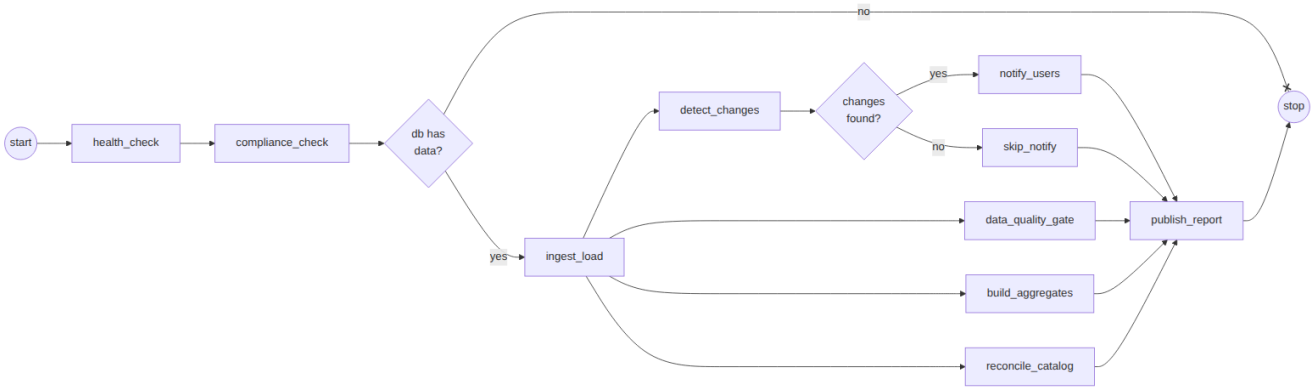


Figure 8 — rendered from the document's Mermaid source

D.7 — ER diagram (warehouse core)

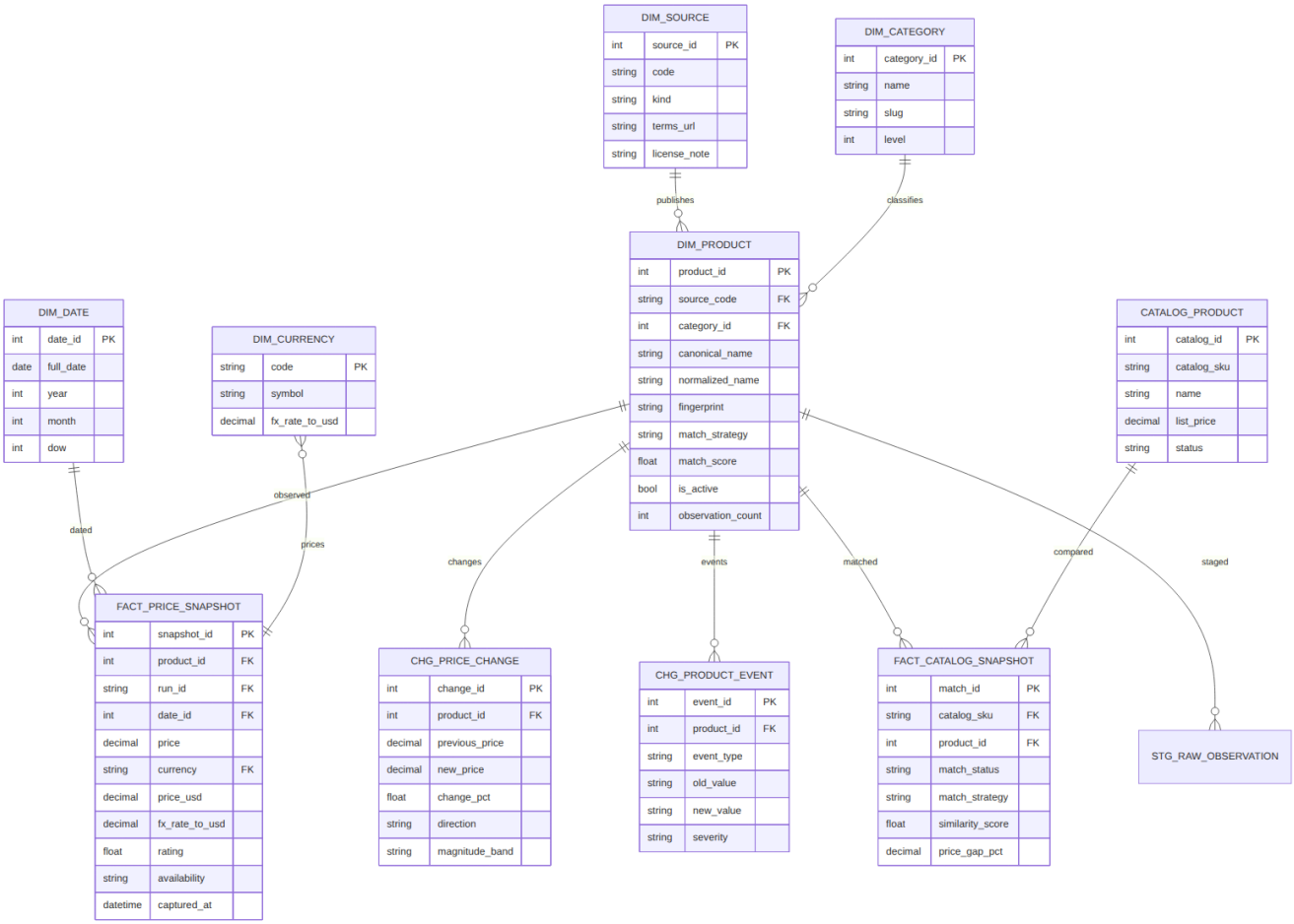


Figure 9 — rendered from the document's Mermaid source

D.8 — Data flow (level 1)

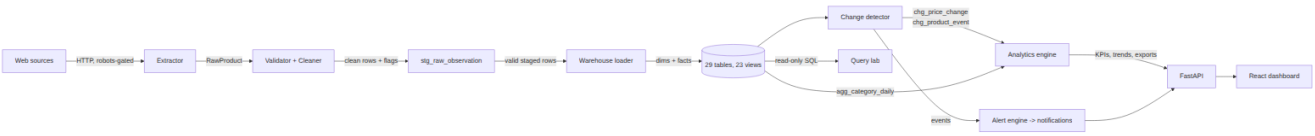


Figure 10 — rendered from the document's Mermaid source

D.9 — Deployment view

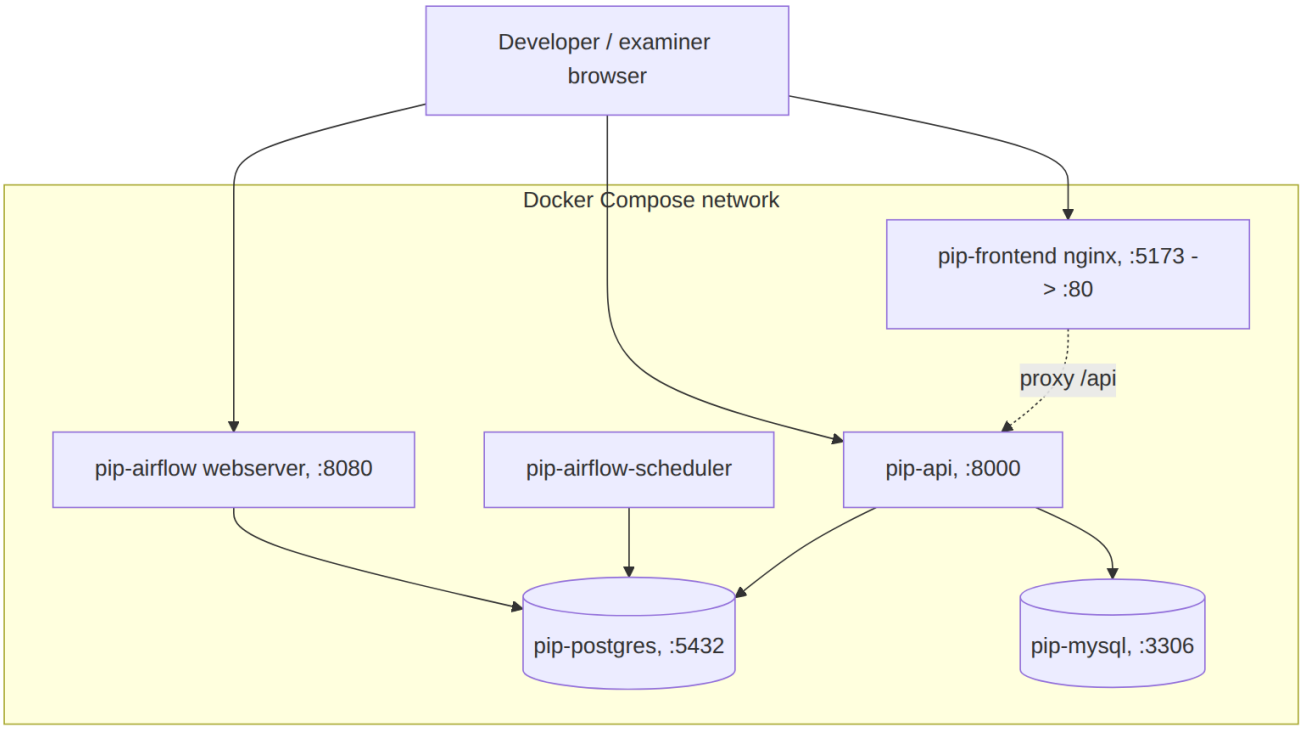


Figure 11 — rendered from the document's Mermaid source

D.10 — The nine stages

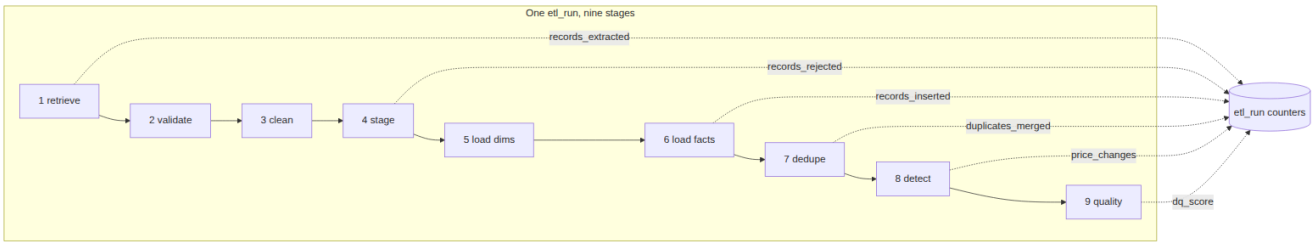


Figure 12 — rendered from the document's Mermaid source

D.11 — Compliance lives in the transport layer

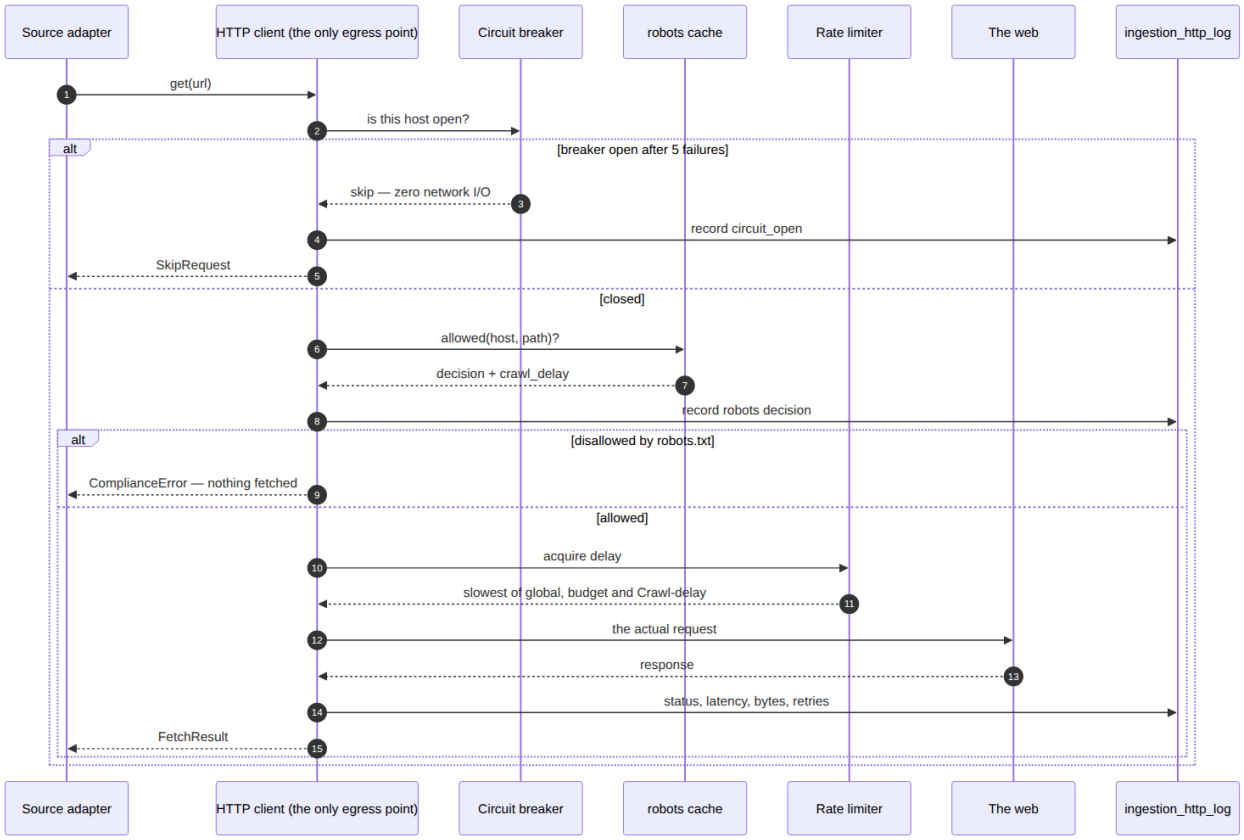


Figure 13 — rendered from the document's Mermaid source

D.12 — Component view

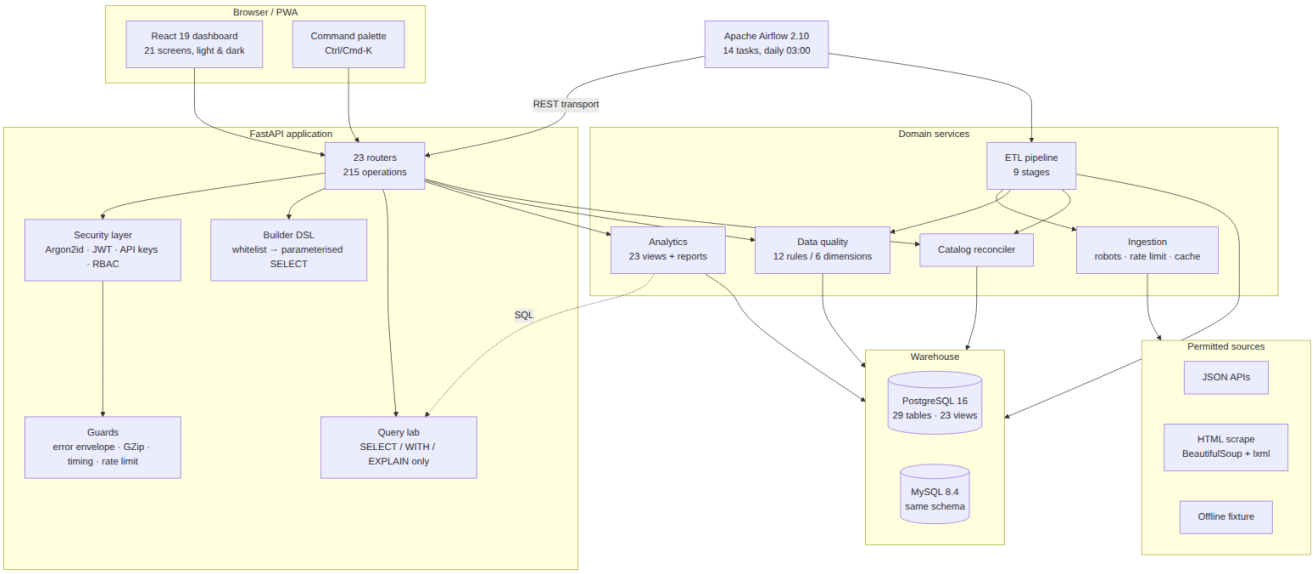


Figure 14 — rendered from the document's Mermaid source

D.13 — Class view of the ingestion contract

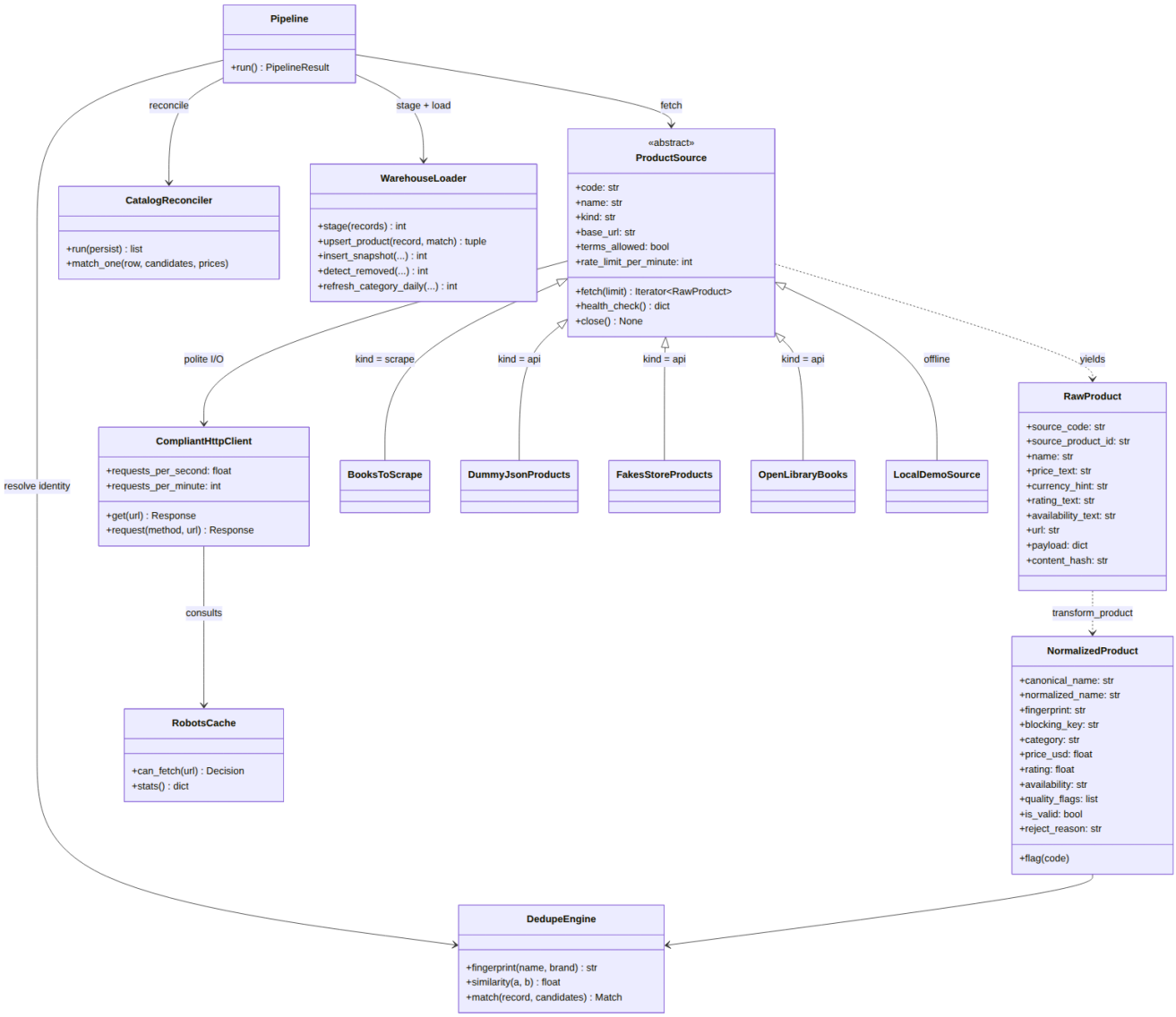


Figure 15 — rendered from the document's Mermaid source

D.14 — State of a pipeline run

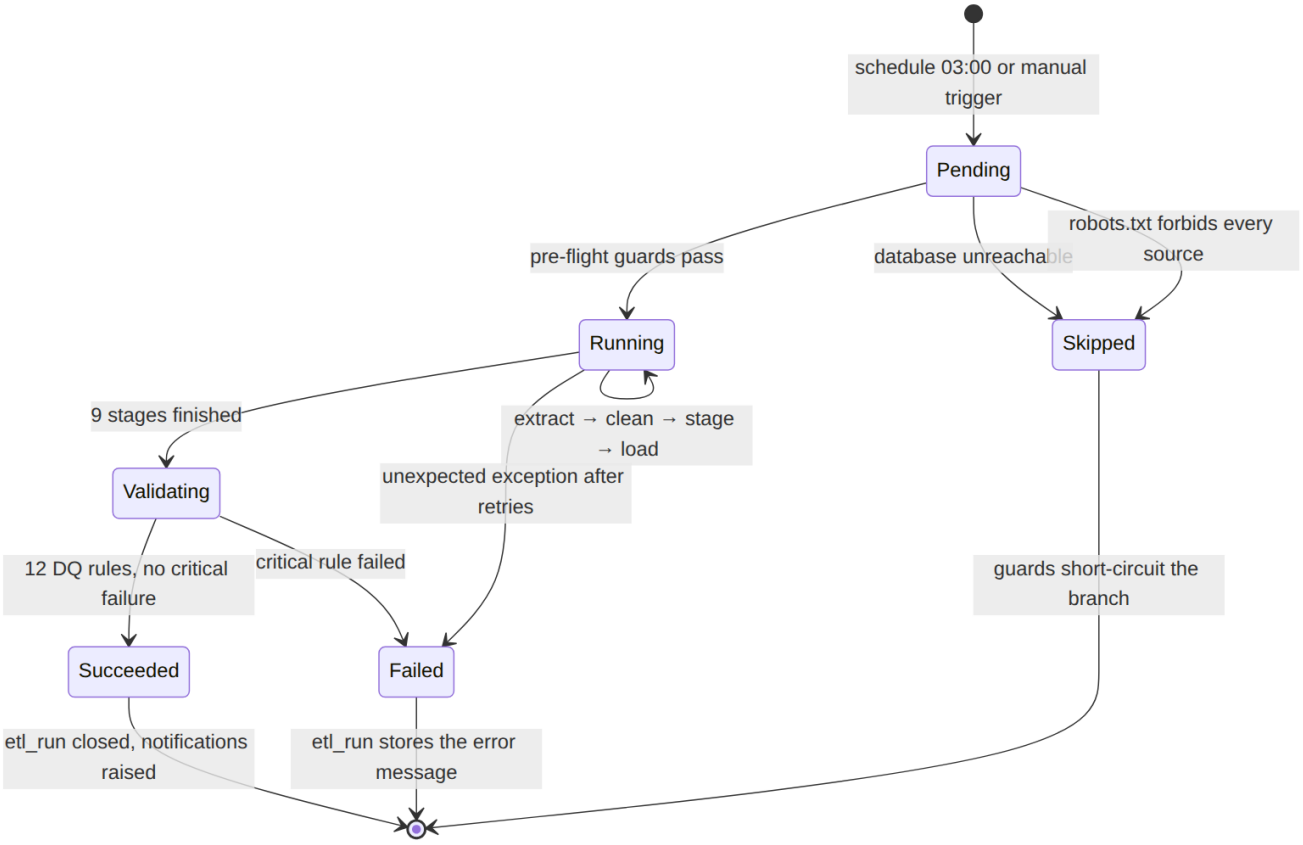


Figure 16 — rendered from the document's Mermaid source

D.15 — Ingestion source catalog

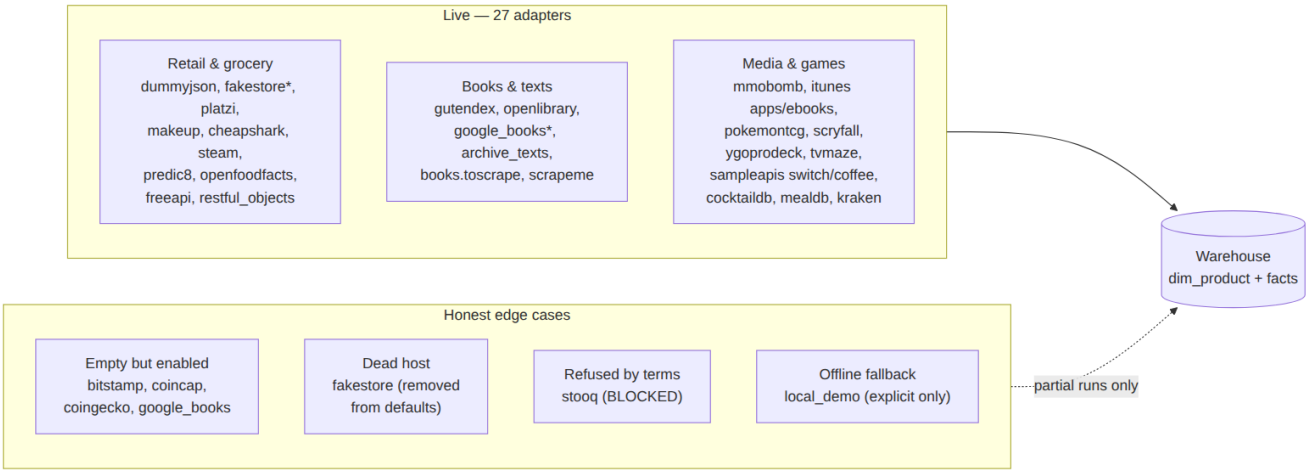


Figure 17 — rendered from the document's Mermaid source

D.16 — Security

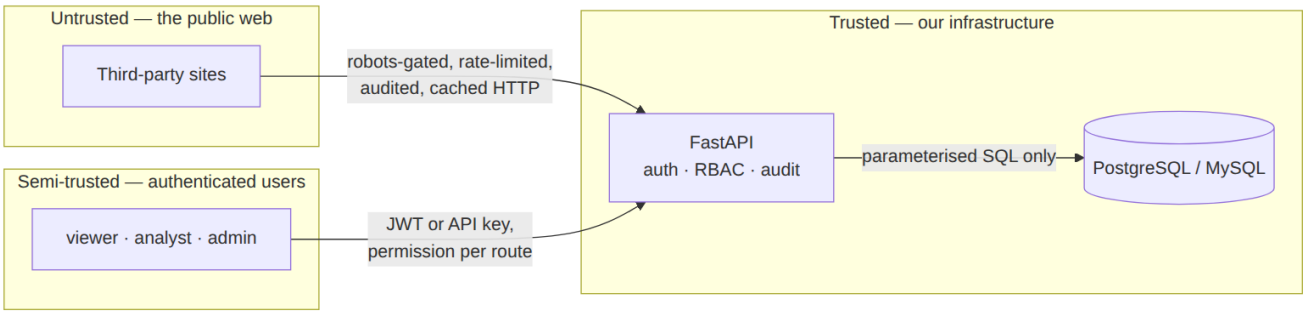


Figure 18 — rendered from the document's Mermaid source